

```
1 # 🎓 RAG AI - Video Lecture Teaching Assistant
2
3 A **Retrieval-Augmented Generation (RAG)** based AI Teaching Assistant that transforms video lectures into an interactive Q&A system. Ask questions about your course material and get accurate, context-aware answers grounded in the actual video content.
4
5 ---
6
7 ## 🚀 Demo
8
9 > Upload a video → Extract audio → Transcribe with Whisper → Chunk & Embed → Ask questions via Claude AI
10
11 ---
12
13 ## 💬 How It Works (RAG Pipeline)
14
15 ```
16 [Video Files]
17     ↓
18 process_videos.py → Extracts audio (.mp3) using FFmpeg
19     ↓
20 translate.py      → Transcribes + translates audio to English using Whisper
21     ↓
22 chunks.py        → Splits transcript into smaller chunks for retrieval
23     ↓
24 [Vector Store]   → Chunks stored as searchable embeddings
25     ↓
26 [RAG Chat App]   → User asks question → Retrieve relevant chunks → Claude answers
27 ```
28
29 ---
30
31 ## 📁 Project Structure
32
33 ```
34 RAG_AI/
35 |— videos/          # Input video files (not tracked in git)
36 |— audios/          # Extracted .mp3 files (not tracked in git)
37 |— jsons/           # Whisper transcription outputs
38 |— process_videos.py # Step 1: Video → Audio extraction
39 |— translate.py      # Step 2: Audio → English transcript
40 |— chunks.py        # Step 3: Transcript → Chunks
41 |— .gitignore
42 ```
43
```

```

44 ---
45
46 ## ⚙️ Tech Stack
47
48 | Tool | Purpose |
49 |-----|-----|
50 | 🐍 Python | Core language |
51 | 🎬 FFmpeg | Video to audio extraction |
52 | 🗣️ OpenAI Whisper | Speech recognition + translation |
53 | 🧩 LangChain | Text chunking + RAG pipeline |
54 | 💬 Claude (Anthropic) | AI answer generation |
55 | 🗄️ FAISS / ChromaDB | Vector storage for embeddings |
56
57 ---
58
59 ## 🛠️ Setup & Installation
60
61 ### 1. Clone the repository
62 ```bash
63 git clone https://github.com/YOUR_USERNAME/RAG_AI.git
64 cd RAG_AI
65 ```
66
67 ### 2. Install dependencies
68 ```bash
69 pip install openai-whisper torch ffmpeg-python langchain anthropic
70 ```
71
72 ### 3. Install FFmpeg
73 - **Windows:** Download from [ffmpeg.org](https://ffmpeg.org/download.html) and add to PATH
74 - **Mac:** `brew install ffmpeg`
75 - **Linux:** `sudo apt install ffmpeg`
76
77 ### 4. Add your video
78 Place your video file inside the `videos/` folder.
79
80 ---
81
82 ## ▶️ Running the Pipeline
83
84 ### Step 1 - Extract audio from video
85 ```bash
86 python process_videos.py
87 ```
88 Converts all videos in `videos/` to `.mp3` files in `audios/`.

```

```

89
90 ### Step 2 - Transcribe & translate
91 ```bash
92 python translate.py
93 ```
94 Uses Whisper (`small` model) to transcribe audio and translate to English. Saves output to `jsons/`.
95
96 ### Step 3 - Chunk the transcript
97 ```bash
98 python chunks.py
99 ```
100 Splits the transcript into smaller overlapping chunks ready for embedding.
101
102 ---
103
104 ## 🧠 Key Design Decisions
105
106 - Whisper `small` model used for local demo; swap to `large-v2` on a GPU server for production-grade accuracy
107 - `enumerate` + `zfill(2)` ensures the pipeline automatically scales to any number of videos without code changes
108 - `exist_ok=True` on folder creation prevents crashes on repeated runs
109 - FFmpeg `-map a` flag extracts only the audio stream, ignoring video, for faster processing
110
111 ---
112
113 ## 📌 Notes
114
115 - Videos and audios are excluded from git (see `.gitignore`) due to large file sizes
116 - JSON transcripts are tracked so reviewers can see pipeline output without running Whisper
117 - To check if GPU acceleration is available for faster Whisper inference:
118 ```bash
119 python -c "import torch; print(torch.cuda.is_available())"
120 ```
121
122 ---
123
124 ## 👤 Author
125
126 **Your Name**
127 [GitHub](https://github.com/YOUR_USERNAME) • [LinkedIn](https://linkedin.com/in/YOUR_USERNAME)
128
129 ---
130
131 ## 📄 License

```

132
133 MIT License - feel free to use and modify for your own projects.
134